The Journal of Open Source Software

# DisCoTec: Distributed higher-dimensional HPC simulations with the sparse grid combination technique

**Theresa Pollinger**[1¶], **Marcel Hurler**[1], **Alexander Van Craen**[1], **Michael Obersteiner**[2], **and Dirk Pflüger**[1]

**1** University of Stuttgart, Scientific Computing, Stuttgart, Germany **2** Technical University of Munich, Chair of Scientific Computing, Munich, Germany ¶ Corresponding author

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:** Daniel S. Katz

**Reviewers:**

- @EmilyBourne
- @jakelangham

**Submitted:** 21 June 2024
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

## Summary

`DisCoTec` is a C++ framework for the sparse grid combination technique, designed for massively parallel settings. It is implemented with shared-memory parallelism via OpenMP and distributed-memory parallelism via MPI, and is intended to be used in conjunction with existing simulation codes. For simulation codes that can handle nested structured grids, little to no adaptation work is needed for use with the `DisCoTec` framework. `DisCoTec` demonstrates its superiority in higher-dimensional time-dependent simulations, such as high-fidelity plasma simulations in 4- to 6-dimensions (Pollinger, Rentrop, et al., 2023) and even for simulations in two dimensions, improvements may be observed.

A central part of the combination technique at scale is the transformation of grid coefficients into a multi-scale basis. `DisCoTec` provides a selection of three different lifting wavelets for this purpose: hierachical hat basis, biorthogonal, and fullweighting basis. In addition, any code that can operate on nested structured grids can benefit from the model order reduction provided by the underlying sparse grid approach used by `DisCoTec`, without requiring any multi-scale operations. An additional feature of `DisCoTec` is the possibility of performing widely-distributed simulations of higher-dimensional problems, where multiple HPC systems collaborate to solve a joint simulation, as demonstrated in (Pollinger, Van Craen, et al., 2023). Thus, `DisCoTec` can leverage the compute power and main memory of multiple HPC systems, with comparatively low and manageable transfer costs due to the combination technique.

## Statement of need

Higher-dimensional problems (by which we mean more than three space dimensions and one time dimension) quickly require infeasible amounts of computational resources such as memory and core-hours as the problem size increases—they are haunted by the so-called 'curse of dimensionality'. An example of this are high-fidelity plasma simulations in the field of confined fusion research. Current approaches to this problem include dimensionally-reduced models (which may not always be applicable), and restricting computations to a very limited resolution. Multi-scale (hierarchical) methods, such as the sparse grid combination technique that `DisCoTec` employs, provide an alternative approach to addressing the curse of dimensionality. While some implementations of the sparse grid combination technique are available in the context of UQ, there is currently no other implementation for parallel simulations that require distributed computing.

`DisCoTec` is a C++ framework for the sparse grid combination technique. Targeted at HPC systems, it is used for parallel simulations (Heene, 2018), drawing on distributed-memory parallelism via MPI and shared-memory parallelism via OpenMP. It is designed to be used in

DRAFT

combination with existing simulation codes, which can be used with `DisCoTec` in a black-box fashion.

## Methods: Sparse grid combination technique and implementation

The sparse grid combination technique (with time-stepping) is a multi-scale approach for solving higher-dimensional problems. Instead of solving the problem on one grid that is very finely resolved in all dimensions, the problem is solved on the so-called 'component grids' which are all rather coarsely resolved—each of them differently in the different dimensions. For instance, the following schematic shows a two-dimensional combination scheme, consisting of seven component grids.

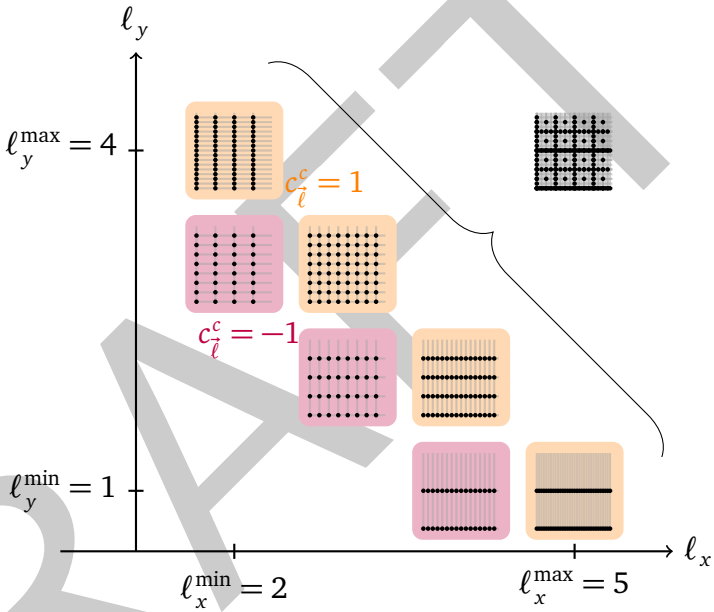


**Figure 1:** Combination scheme in two dimensions with $\vec{l}_{min} = (2, 1)$ and $\vec{l}_{max} = (5, 4)$, periodic boundary conditions. Figure first published in (Pollinger, 2024).

By updating each other's information throughout the simulation, the component grids still obtain an accurate solution of the overall problem (Griebel et al., 1992). This is enabled by an intermedate transformation into a multi-scale (hierarchical) basis, and application of the combination formula

$$f^{(\mathrm{s})} = \sum_{\vec{l} \in \mathcal{I}} c_{\vec{l}} f_{\vec{l}}$$

where $f^{(\mathrm{s})}$ is the sparse grid approximation, and $f_{\vec{l}}$ are the component grid functions. In Figure 1, the coefficients $c_{\vec{l}}$ are $-1$ for the coarser component grids (red background) and $1$ for the finer component grids (orange background). In summary, each of the grids will run (one or more) time steps of the simulation, then exchange information with the other grids, and repeat this process until the simulation is finished.

`DisCoTec` provides the necessary infrastructure for the combination technique with a black-box approach, enabling massive parallelism—suitable for existing distributed solvers that use structured grids. An important feature is the usage of 'process groups', where multiple MPI ranks will collaborate on a set of component grids, and the solver's existing parallelism can be re-used. The process groups are displayed as $pg_i$ in Figure 2.

DRAFT

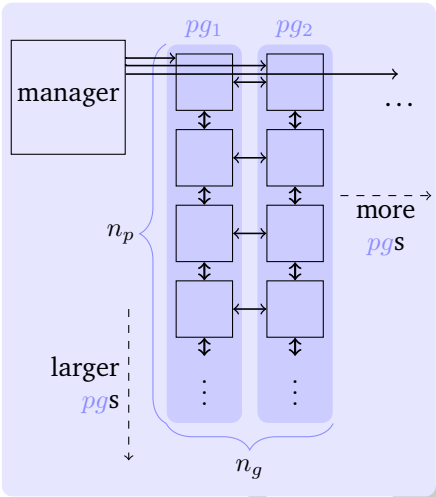


**Figure 2:** `DisCoTec` process groups: Each black square denotes one MPI rank. The ranks are grouped into the so-called 'process groups'. Distributed operations in `DisCoTec` require either communication in the process group, or perpendicular to it—there is no need for global communication or synchronization, which avoids a major scaling bottleneck. The manager rank is optional. Figure first published in (Pollinger, 2024).

In addition, the number of process groups can be increased to leverage the combination technique's embarrassing parallelism in the solver time steps. In Figure 2, this would be equivalent to adding more and more process groups to the right.

Using `DisCoTec`, kinetic simulations were demonstrated to scale up to hundreds of thousands of CPU cores (Pollinger, 2024). By putting a special focus on saving memory, most of the memory is available for use by the black-box solver, even at high core counts. In addition, OpenMP parallelism can be used to further increase parallelism and to decrease main memory usage.

Through highly parallel I/O operations, `DisCoTec` can be used to perform simulations on multiple HPC systems simultaneously, if there exists a tool for sufficiently fast file transfer between the systems (Pollinger, Van Craen, et al., 2023). The `DisCoTec` repository contains example scripts and documentation for utilizing UFTP as an example of a transfer tool, but the approach is not limited to UFTP.

`DisCoTec` provides a conveniently automated way of installing using a spack package (Gamblin et al., 2015).

# State of the field

Besides `DisCoTec` there exist other frameworks that allow the usage of sparse grids and the combination technique. We will give a brief overview and outline the differences and application areas of the codes.

The C++ code SG++ allows to directly utilize sparse grids and apply them to a variety of different tasks such as interpolation, quadrature, optimization, PDEs, regression, and classification. With the help of wrappers, the framework can be used from various other programming languages such as Python and Matlab. The code targets direct implementations within sparse grids and provides a basic implementation of the combination technique. Although offering parallelization for some of the tasks, the code mainly targets single-node computations.

The `Sparse Grids Matlab Kit` by Piazzola and Tamellini was originally designed for teaching purposes and uncertainty quantification with the combination technique (Piazzola & Tamellini,

2022). It offers a user friendly MATLAB interface for the combination technique. In addition, dimensional adaptivity is available for nested and non-nested sequences of component grid collocation points. The code is designed for usage on a single node which limits the parallelism to shared memory.

The `sparseSpACE` project offers different variants of the combination technique including a spatially adaptive combination technique. It provides implementations for various applications such as numerical integration, interpolation, uncertainty quantification, sparse grid density estimation (for classification and clustering), regression, and PDE calculations. The code is completely written in Python and is mostly sequential. The main novelty of this project is the possibility to add spatial adaptivity to the combination technique.

This demonstrates that there exist multiple codes for sparse grids and the combination technique. However, `DisCoTec` is the only code that offers distributed parallelization with the combination technique and has demonstrated that it can scale up to full supercomputers and beyond. In addition, `DisCoTec` uses the most sophisticated approach to utilize the combination technique with time-dependent PDEs by employing recombinations, which increases the overall numerical accuracy (Pollinger, Rentrop, et al., 2023).

# Acknowledgements

We acknowledge contributions from Mario Heene, Christoph Kowitz, Alfredo Parra Hinojosa, Johannes Rentrop, Keerthi Gaddameedi, Marvin Dostal, Marcel Breyer, Christoph Niethammer, Philipp Offenhäuser, and support from HLRS, LRZ, JSC, and NHR@FAU, where we would like to highlight the long-standing support by Martin Bernreuther and Martin Ohlerich in particular.

# References

Gamblin, T., LeGendre, M., Collette, M. R., Lee, G. L., Moody, A., Supinski, B. R. de, & Futral, S. (2015). *The Spack package manager: Bringing order to HPC software chaos*. 1–12. https://doi.org/10.1145/2807591.2807623

Griebel, M., Schneider, M., & Zenger, C. (1992). A combination technique for the solution of sparse grid problems. In P. de Groen & R. Beauwens (Eds.), *Iterative Methods in Linear Algebra* (pp. 263–281). IMACS, Elsevier, North Holland.

Heene, M. (2018). *A Massively Parallel Combination Technique for the Solution of High-Dimensional PDEs* [PhD thesis, Universität Stuttgart]. https://doi.org/10.18419/opus-9893

Obersteiner, M. (2021). *A spatially adaptive and massively parallel implementation of the fault-tolerant combination technique* [PhD thesis, Technische Universität München]. https://mediatum.ub.tum.de/doc/1613369/1613369.pdf

Piazzola, C., & Tamellini, L. (2022, March 17). *The Sparse Grids Matlab kit – a Matlab implementation of sparse grids for high-dimensional function approximation and uncertainty quantification*. https://doi.org/10.48550/arXiv.2203.09314

Pollinger, T. (2024). *Stable and mass-conserving high-dimensional simulations with the sparse grid combination technique for full HPC systems and beyond* [PhD thesis]. https://doi.org/10.18419/opus-14210

Pollinger, T., Rentrop, J., Pflüger, D., & Kormann, K. (2023). A stable and mass-conserving sparse grid combination technique with biorthogonal hierarchical basis functions for kinetic simulations. *Journal of Computational Physics*, 112338. https://doi.org/10.1016/j.jcp.2023.112338

Pollinger, T., Van Craen, A., Niethammer, C., Breyer, M., & Pflüger, D. (2023, November 11).

Leveraging the Compute Power of Two HPC Systems for Higher-Dimensional Grid-Based Simulations with the Widely-Distributed Sparse Grid Combination Technique. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. https://doi.org/10.1145/3581784.3607036

DRAFT